

SQL Syntax Reference

Every clause, operator and function you need for a PostgreSQL interview — each one with what it does in a single line, the syntax, a worked example, and the answer that example actually returns.

337

entries

28

sections

300

live examples

PG 18

dialect

How to read an entry. The left column is the syntax. The right column is what it does, then the example, then → the real result. Every example was executed against the sql-prep teaching database while this document was generated — the results are recorded output, not illustrations.

- | | |
|---|--|
| 1 How a query is read | 15 ORDER BY, LIMIT & OFFSET |
| 2 Punctuation, literals & comments | 16 JOINS |
| 3 SELECT & FROM | 17 Set operations |
| 4 Comparison operators | 18 Subqueries |
| 5 Pattern matching | 19 CTEs (WITH clause) |
| 6 Logical operators | 20 Window functions |
| 7 Arithmetic operators | 21 DDL — defining tables |
| 8 Data types & casting | 22 DML — changing data |
| 9 Conditional expressions | 23 Transactions & concurrency |
| 10 String functions | 24 Views, functions & triggers |
| 11 Numeric functions | 25 Performance & inspection |
| 12 Date & time functions | 26 How NULL behaves, everywhere |
| 13 Aggregate functions | 27 Interview patterns |
| 14 GROUP BY & HAVING | 28 Dialect differences |

1 · How a query is read

You write a query in one order; the database evaluates it in another. Almost every "why does this not work?" is answered here.

Written order	The order you must TYPE the clauses in. Getting this wrong is a syntax error. SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → WINDOW → ORDER BY → LIMIT
Execution order	The order the database actually RUNS them in. This is what governs what is visible where. FROM → JOIN/ON → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
Why an alias fails in WHERE	WHERE runs at step 3, SELECT at step 6 — the alias does not exist yet. SELECT salary*12 AS annual ... WHERE annual > 5 × ... ORDER BY annual ✓
Grain	What one row of a result represents. Say it out loud before writing anything. SELECT COUNT(*) AS line_rows, COUNT(DISTINCT o.order_id) AS real_orders FROM orders o JOIN order_items oi ON oi.order_id=o.order_id → line_rows=119 real_orders=48

2 · Punctuation, literals & comments

The mechanical rules: where commas go, which quote means what, when brackets are required.

, (comma)	Separates items in a LIST — columns, tables, values, GROUP BY keys. Never before FROM or before a closing bracket. SELECT cust_name, country, segment FROM customers LIMIT 2 → cust_name=Zenith Retail country=India segment=Enterprise · cust_name=Lumen Analytics country=India segment=Mid-Market (2 rows)
Missing comma	Does NOT error — SQL reads the next word as an alias. Silently returns fewer columns. SELECT emp_name title FROM employees LIMIT 2 → title=Rohan Mehta · title=Priya Nair (2 rows)
' ' (single quotes)	Wraps a STRING LITERAL — a piece of data. SELECT * FROM customers WHERE country = 'India' LIMIT 2 → cust_id=1 cust_name=Zenith Retail email=ops@zenith.example city=Bengaluru country=India segment=Enterprise signup_date=2023-01-15 (2 rows)
" " (double quotes)	Wraps an IDENTIFIER (table/column name) and makes it case-sensitive. Data in double quotes is an error. SELECT cust_name AS "Customer Name" FROM customers LIMIT 2 → Customer Name=Zenith Retail · Customer Name=Lumen Analytics (2 rows)
'' (doubled quote)	Escapes an apostrophe inside a string literal. SELECT '0'Brien' AS name → 0'Brien
\$\$... \$\$	Dollar quoting — a string containing quotes, with no escaping needed. SELECT \$\$0'Brien\$\$ AS name → 0'Brien
; (semicolon)	Ends a statement. Required when sending several statements at once. SELECT 1; SELECT 2;
-- comment	Comment to end of line. SELECT 1 AS x -- everything after this is ignored → 1
/* ... */	Block comment; may be nested. SELECT /* inline note */ 1 AS x → 1

() required	Function calls, IN lists, every subquery, INSERT/CREATE column lists, and OVER (...). <code>SELECT COUNT(*) FROM orders WHERE status IN ('completed','pending')</code> → 41
() advised	Around an OR group whenever AND and OR are mixed — AND binds tighter than OR. <code>SELECT COUNT(*) AS n FROM orders WHERE (status='completed' OR status='pending') AND channel='web'</code> → 8
Case folding	Keywords are case-insensitive. Unquoted identifiers fold to lower case. String COMPARISON is case-sensitive. <code>SELECT 'Mumbai'='mumbai' AS exact, 'Mumbai' ILIKE 'mumbai' AS insensitive</code> → <code>exact=false insensitive=true</code>

3 · SELECT & FROM

Choosing what comes out, and where rows come from.

SELECT expr, ...	Lists the output columns. Columns, expressions, literals and function calls all allowed. <code>SELECT emp_name, salary, salary*12 AS annual, 'INR' AS currency FROM employees LIMIT 3</code> → <code>emp_name=Rohan Mehta salary=250000.00 annual=3000000.00 currency=INR · emp_name=Priya Nair salary=190000.00 annual=2280000.00 currency=INR ... 3 rows</code>
AS alias	Renames a column or table in the output. Optional keyword, but always write it. <code>SELECT emp_name AS name, salary AS pay FROM employees LIMIT 2</code> → <code>name=Rohan Mehta pay=250000.00 · name=Priya Nair pay=190000.00 (2 rows)</code>
SELECT *	Every column. Fine while exploring; a liability in production code. <code>SELECT * FROM departments LIMIT 2</code> → <code>dept_id=1 dept_name=Engineering location=Bengaluru budget=2000000.00 · dept_id=2 dept_name=Sales location=Mumbai budget=1200000.00 (2 rows)</code>
SELECT DISTINCT	Removes duplicate OUTPUT ROWS (the whole row, not one column). <code>SELECT DISTINCT country FROM customers ORDER BY country</code> → <code>country=Germany · country=India ... 5 rows</code>
DISTINCT ON (cols)	PostgreSQL only. Keeps the FIRST row per group given an ORDER BY — "latest record per customer" in one line. <code>SELECT DISTINCT ON (cust_id) cust_id, order_id, order_date FROM orders ORDER BY cust_id, order_date DESC</code> → <code>cust_id=1 order_id=1040 order_date=2024-06-15 · cust_id=2 order_id=1039 order_date=2024-06-03 ... 11 rows</code>
FROM table alias	The starting row source. An alias shortens every later reference. <code>SELECT e.emp_name FROM employees e LIMIT 2</code> → <code>emp_name=Rohan Mehta · emp_name=Priya Nair (2 rows)</code>
FROM (subquery) AS t	A derived table. The alias is MANDATORY in PostgreSQL. <code>SELECT t.dept_id, t.n FROM (SELECT dept_id, COUNT(*) AS n FROM employees GROUP BY dept_id) AS t ORDER BY t.n DESC LIMIT 3</code> → <code>dept_id=1 n=8 · dept_id=2 n=4 ... 3 rows</code>
FROM (VALUES ...) AS t(c)	A literal inline table — useful for lookup/banding tables. <code>SELECT * FROM (VALUES ('a',1),('b',2)) AS v(letter, num)</code> → <code>letter=a num=1 · letter=b num=2 (2 rows)</code>
generate_series(a,b,step)	A set-returning function used as a table — builds number and date spines. <code>SELECT g::date AS month FROM generate_series(DATE '2024-01-01', DATE '2024-04-01', INTERVAL '1 month') g</code> → <code>month=2024-01-01 · month=2024-02-01 ... 4 rows</code>

4 · Comparison operators

Every one of these returns *TRUE*, *FALSE* or *NULL*. *WHERE* keeps a row only on *TRUE*.

=	Equal to. <pre>SELECT COUNT(*) AS n FROM orders WHERE status = 'completed'</pre> → 37
<>	Not equal to. The SQL standard spelling. <pre>SELECT COUNT(*) AS n FROM orders WHERE status <> 'completed'</pre> → 11
!=	Not equal to. Identical to <> in PostgreSQL; <> is the portable one. <pre>SELECT COUNT(*) AS n FROM orders WHERE status != 'cancelled'</pre> → 45
< >	Less than / greater than. <pre>SELECT COUNT(*) AS n FROM employees WHERE salary > 120000</pre> → 9
<= >=	Less than or equal / greater than or equal. <pre>SELECT COUNT(*) AS n FROM employees WHERE salary >= 150000</pre> → 7
BETWEEN a AND b	Range test, INCLUSIVE of both ends. Same as a <= x AND x <= b. <pre>SELECT COUNT(*) AS n FROM employees WHERE salary BETWEEN 100000 AND 150000</pre> → 8
NOT BETWEEN a AND b	Outside the range, exclusive of both ends. <pre>SELECT COUNT(*) AS n FROM employees WHERE salary NOT BETWEEN 100000 AND 150000</pre> → 8
IN (list)	Matches any value in a list or subquery. <pre>SELECT COUNT(*) AS n FROM customers WHERE country IN ('India','UK')</pre> → 8
NOT IN (list)	Matches none of them. DANGER: returns zero rows if the list contains a NULL. <pre>SELECT COUNT(*) AS n FROM customers WHERE country NOT IN ('India','UK')</pre> → 6
IS NULL	Tests for a missing value. The ONLY correct way — = NULL is never true. <pre>SELECT COUNT(*) AS n FROM customers WHERE city IS NULL</pre> → 2
IS NOT NULL	Tests that a value is present. <pre>SELECT COUNT(*) AS n FROM customers WHERE city IS NOT NULL</pre> → 12
IS DISTINCT FROM	Like <>, but NULL-safe: treats NULL as a comparable value. <pre>SELECT (NULL IS DISTINCT FROM NULL) AS both_null, (1 IS DISTINCT FROM NULL) AS one_null</pre> → both_null=false one_null=true
IS NOT DISTINCT FROM	Like =, but NULL-safe. NULL equals NULL here. <pre>SELECT (NULL IS NOT DISTINCT FROM NULL) AS equal_nulls</pre> → true
IS TRUE / IS FALSE	Tests a boolean, treating NULL as not-true rather than unknown. <pre>SELECT (NULL IS TRUE) AS null_is_true, (false IS FALSE) AS false_is_false</pre> → null_is_true=false false_is_false=true

EXISTS (subquery)	TRUE if the subquery returns at least one row. Stops at the first match. <pre>SELECT COUNT(*) AS n FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.cust_id=c.cust_id)</pre> → 11
NOT EXISTS (subquery)	TRUE if the subquery returns nothing. The NULL-safe way to write an anti-join. <pre>SELECT COUNT(*) AS n FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cust_id=c.cust_id)</pre> → 3
x > ANY (subquery)	True if x beats AT LEAST ONE value — i.e. greater than the minimum. ANY = SOME. <pre>SELECT COUNT(*) AS n FROM employees WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id=3)</pre> → 15
x > ALL (subquery)	True if x beats EVERY value — i.e. greater than the maximum. <pre>SELECT COUNT(*) AS n FROM employees WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id=3)</pre> → 3

5 · Pattern matching

Text search inside WHERE.

LIKE 'A%'	Pattern match. % = any number of characters, _ = exactly one character. <pre>SELECT COUNT(*) AS n FROM customers WHERE cust_name LIKE 'A%'</pre> → 1
LIKE '%data%'	Contains. A LEADING % prevents the use of a normal index. <pre>SELECT COUNT(*) AS n FROM products WHERE product_name LIKE '%Data%'</pre> → 1
LIKE '_a%'	Underscore matches exactly one character — here, second letter is "a". <pre>SELECT COUNT(*) AS n FROM customers WHERE cust_name LIKE '_a%'</pre> → 2
NOT LIKE	Does not match the pattern. <pre>SELECT COUNT(*) AS n FROM customers WHERE cust_name NOT LIKE 'A%'</pre> → 13
ILIKE	PostgreSQL: case-insensitive LIKE. <pre>SELECT COUNT(*) AS n FROM customers WHERE cust_name ILIKE '%0%'</pre> → 9
ESCAPE '\'	Lets you search for a literal % or _ character. <pre>SELECT '100%' LIKE '100\%' AS matches_literal_percent</pre> → true
~ ~*	PostgreSQL regex match / case-insensitive regex match. <pre>SELECT 'abc123' ~ '[0-9]+\$' AS ends_with_digits</pre> → true
!~ !~*	Regex does NOT match / case-insensitively. <pre>SELECT 'abc' !~ '[0-9]' AS has_no_digit</pre> → true
SIMILAR TO	SQL-standard pattern language, halfway between LIKE and regex. Rarely worth it. <pre>SELECT 'abc' SIMILAR TO '(a z)%' AS matches</pre> → true

6 · Logical operators

Combining conditions. AND binds tighter than OR — bracket whenever you mix them.

AND	Both sides must be true. <code>SELECT COUNT(*) AS n FROM orders WHERE status='completed' AND channel='web'</code> → 8
OR	Either side may be true. <code>SELECT COUNT(*) AS n FROM orders WHERE status='cancelled' OR status='refunded'</code> → 7
NOT	Inverts a condition. NOT UNKNOWN is still UNKNOWN. <code>SELECT COUNT(*) AS n FROM orders WHERE NOT (status='completed')</code> → 11
Precedence	Highest to lowest: :: → unary - → * / % → + - → → BETWEEN/IN/LIKE → comparisons → IS → NOT → AND → OR. a OR b AND c == a OR (b AND c)
Three-valued logic	TRUE AND NULL = NULL. FALSE AND NULL = FALSE. TRUE OR NULL = TRUE. FALSE OR NULL = NULL. <code>SELECT (true AND NULL) AS t_and_n, (false AND NULL) AS f_and_n, (true OR NULL) AS t_or_n, (false OR NULL) AS f_or_n</code> → t_and_n=NULL f_and_n=false t_or_n=true f_or_n=NULL

7 · Arithmetic operators

+ - * /	Add, subtract, multiply, divide. <code>SELECT 10+3 AS add, 10-3 AS sub, 10*3 AS mul, 10.0/3 AS div</code> → add=13 sub=7 mul=30 div=3.3333333333333333
/ on integers	TRUNCATES — 7/2 is 3, not 3.5. Cast one side to numeric to get a decimal. <code>SELECT 7/2 AS integer_div, 7/2.0 AS decimal_div, 7::numeric/2 AS cast_div</code> → integer_div=3 decimal_div=3.5000000000000000 cast_div=3.5000000000000000
%	Modulo — the remainder after division. <code>SELECT 17 % 5 AS remainder</code> → 2
^	Exponentiation. <code>SELECT 2 ^ 10 AS power</code> → 1024
/ /	Square root / cube root operators. <code>SELECT / 144 AS sqrt, / 27 AS cbrt</code> → sqrt=12 cbrt=3
@	Absolute value operator. <code>SELECT @ -17 AS absolute</code> → 17

8 · Data types & casting

What to store a value as, and how to convert between them.

integer / bigint / smallint	Whole numbers. bigint for large IDs, integer for most things. <code>SELECT 42::integer AS i, 42::bigint AS b</code> → i=42 b=42
-----------------------------	---

numeric(p,s) / decimal	EXACT decimal. The correct type for money — always. <pre>SELECT 1234.5678::numeric(8,2) AS money → 1234.57</pre>
real / double precision	Approximate floating point. Fast, but never use for money. <pre>SELECT 0.1::float8+0.2::float8 AS float_sum, 0.1::numeric+0.2::numeric AS numeric_sum → float_sum=0.30000000000000004 numeric_sum=0.3</pre>
text / varchar(n) / char(n)	Strings. text is the PostgreSQL default; varchar(n) only when a length limit is a real rule. <pre>SELECT 'abc'::text AS t, 'abc'::varchar(5) AS v → t=abc v=abc</pre>
boolean	TRUE / FALSE / NULL. <pre>SELECT true AS t, false AS f, NULL::boolean AS unknown → t=true f=false unknown=NULL</pre>
date	A calendar day, no time. <pre>SELECT DATE '2024-05-15' AS d → 2024-05-15</pre>
timestamp	Date and time with NO time zone. <pre>SELECT TIMESTAMP '2024-05-15 14:30:00' AS ts → 2024-05-15</pre>
timestampz	Date and time, zone-aware. Use this for anything real-world. <pre>SELECT TIMESTAMPTZ '2024-05-15 14:30:00+05:30' AS tstz → 2024-05-15</pre>
interval	A duration. Add or subtract it from a date. <pre>SELECT INTERVAL '1 month 3 days' AS span → 1 mon 3 days</pre>
uuid	128-bit identifier, for distributed keys. <pre>SELECT '11111111-2222-3333-4444-555555555555'::uuid AS id → 11111111-2222-3333-4444-555555555555</pre>
jsonb	Binary JSON — indexable and queryable, unlike plain json. <pre>SELECT '{"a":1}'::jsonb -> 'a' AS value → 1</pre>
array type[]	An ordered list in one column. PostgreSQL-specific. <pre>SELECT ARRAY['a','b','c'] AS tags, (ARRAY['a','b','c'])[2] AS second → tags=["a","b","c"] second=b</pre>
CAST(x AS type)	Convert a value to another type. ANSI standard, works in every dialect. <pre>SELECT CAST('42' AS integer) AS n → 42</pre>
x::type	PostgreSQL shorthand for CAST. Shorter, but not portable. <pre>SELECT '2024-05-15'::date AS d, '42'::int AS n → d=2024-05-15 n=42</pre>
GENERATED ... AS IDENTITY	Modern auto-incrementing key. Preferred over serial. <pre>id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY</pre>
serial / bigserial	Legacy auto-increment. Still common in older schemas. <pre>id serial PRIMARY KEY</pre>

9 · Conditional expressions

if/else inside SQL, and the NULL toolkit.

CASE WHEN ... THEN ... ELSE ... END	Searched CASE: evaluates conditions in order, returns the first match. Omit ELSE and unmatched rows get NULL. <pre>SELECT emp_name, CASE WHEN salary>=175000 THEN 'exec' WHEN salary>=130000 THEN 'senior' ELSE 'other' END AS band FROM employees LIMIT 4</pre> → emp_name=Rohan Mehta band=exec · emp_name=Priya Nair band=exec ... 4 rows
CASE x WHEN v THEN ... END	Simple CASE: shorthand for equality against one expression only. <pre>SELECT title, CASE title WHEN 'CEO' THEN 'founder' ELSE 'staff' END AS kind FROM employees LIMIT 3</pre> → title=CEO kind=founder · title=VP Engineering kind=staff ... 3 rows
COALESCE(a,b,c)	Returns the first argument that is not NULL. <pre>SELECT COALESCE(NULL, NULL, 'fallback') AS first_non_null</pre> → fallback
NULLIF(a,b)	Returns NULL if a equals b, else a. The standard divide-by-zero guard. <pre>SELECT 10/NULLIF(0,0) AS safe_divide, NULLIF(5,5) AS same, NULLIF(5,9) AS different</pre> → safe_divide=NULL same=NULL different=5
GREATEST(a,b,...)	Largest of several values ACROSS COLUMNS (not down rows). Skips NULLs. <pre>SELECT GREATEST(3,9,2) AS biggest</pre> → 9
LEAST(a,b,...)	Smallest of several values across columns. Skips NULLs. <pre>SELECT LEAST(3,9,2) AS smallest</pre> → 2

10 · String functions

Everything you might be asked to do to text.

a b	Concatenate. If EITHER side is NULL the whole result is NULL. <pre>SELECT 'abc' 'def' AS joined, 'abc' NULL AS with_null</pre> → joined=abcdef with_null=NULL
CONCAT(a,b,...)	Concatenate, treating NULL as an empty string. <pre>SELECT CONCAT('abc', NULL, 'def') AS joined</pre> → abcdef
CONCAT_WS(sep,a,b,...)	Concatenate with a separator, SKIPPING NULLs. Best for building addresses. <pre>SELECT CONCAT_WS(' ', 'Mumbai', NULL, 'India') AS location</pre> → Mumbai, India
LENGTH(s) / CHAR_LENGTH(s)	Number of characters. <pre>SELECT LENGTH('hello') AS len</pre> → 5
UPPER(s) / LOWER(s)	Convert case. <pre>SELECT UPPER('abc') AS up, LOWER('ABC') AS low</pre> → up=ABC low=abc
INITCAP(s)	Capitalises the first letter of each word. <pre>SELECT INITCAP('zenith retail ltd') AS titled</pre> → Zenith Retail Ltd
REPLACE(s, from, to)	Replaces EVERY occurrence of one substring with another. Case-sensitive. <pre>SELECT REPLACE('2024-01-05', '-', '/') AS swapped</pre> → 2024/01/05

TRANSLATE(s, from, to)	Replaces individual CHARACTERS one-for-one, like a cipher. <code>SELECT TRANSLATE('2024-01-05', '-0', '/0') AS translated</code> → <code>2024/01/05</code>
SUBSTRING(s FROM p FOR n)	n characters starting at position p (1-based). Also SUBSTRING(s, p, n). <code>SELECT SUBSTRING('abcdef' FROM 2 FOR 3) AS part, SUBSTRING('abcdef', 2, 3) AS same</code> → <code>part=bcd same=bcd</code>
LEFT(s,n) / RIGHT(s,n)	First / last n characters. <code>SELECT LEFT('abcdef',3) AS l, RIGHT('abcdef',3) AS r</code> → <code>l=abc r=def</code>
TRIM(s)	Removes leading and trailing whitespace (or given characters). <code>SELECT '[' TRIM(' abc ') ']' AS trimmed</code> → <code>[abc]</code>
LTRIM(s) / RTRIM(s)	Trims only the left / only the right. <code>SELECT '[' LTRIM(' abc') ']' AS l, '[' RTRIM('abc ') ']' AS r</code> → <code>l=[abc] r=[abc]</code>
BTRIM(s, chars)	Trims the given characters from both ends. <code>SELECT BTRIM('xabcxx','x') AS trimmed</code> → <code>abc</code>
LPAD(s,n,c) / RPAD(s,n,c)	Pads to width n with character c. Used for fixed-width codes. <code>SELECT LPAD('7',3,'0') AS padded, RPAD('7',3,'-') AS right_padded</code> → <code>padded=007 right_padded=7--</code>
POSITION(sub IN s)	1-based index of a substring; 0 if absent. <code>SELECT POSITION('cd' IN 'abcdef') AS idx</code> → <code>3</code>
STRPOS(s, sub)	Same as POSITION with the arguments the other way round. <code>SELECT STRPOS('abcdef','cd') AS idx</code> → <code>3</code>
SPLIT_PART(s, delim, n)	Returns the nth field after splitting. Ideal for emails, paths and codes. <code>SELECT SPLIT_PART('ops@zenith.example','@',2) AS domain</code> → <code>zenith.example</code>
REPEAT(s,n)	Repeats a string n times — used for indenting tree output. <code>SELECT REPEAT('ab',3) AS repeated</code> → <code>ababab</code>
REVERSE(s)	Reverses the characters. <code>SELECT REVERSE('abc') AS backwards</code> → <code>cba</code>
STARTS_WITH(s, prefix)	Boolean prefix test; can use an index, unlike a regex. <code>SELECT STARTS_WITH('abcdef','abc') AS yes</code> → <code>true</code>
FORMAT(fmt, args...)	printf-style string building. %s = value, %l = identifier, %L = quoted literal. <code>SELECT FORMAT('Hi %s, you are %s', 'Asha', 30) AS greeting</code> → <code>Hi Asha, you are 30</code>
REGEXP_REPLACE(s,pat,repl[,flags])	Regex substitution. Add flag "g" to replace every match, not just the first. <code>SELECT REGEXP_REPLACE('a1b2c3','[0-9]','\#','g') AS masked</code> → <code>a#b#c#</code>

REGEXP_MATCHES(s,pat)	Returns captured groups as an array; with "g" returns one row per match. SELECT (REGEXP_MATCHES('order-1234','([a-z]+)-([0-9]+)'))[2] AS number → 1234
REGEXP_SPLIT_TO_ARRAY(s,pat)	Splits a string by a regex into an array. SELECT REGEXP_SPLIT_TO_ARRAY('a1b22c','[0-9]+') AS parts → ["a","b","c"]
MD5(s)	Hex MD5 hash — handy for building a change-detection fingerprint. SELECT LEFT(MD5('abc'),12) AS hash_prefix → 900150983cd2
ASCII(c) / CHR(n)	Character to code point and back. SELECT ASCII('A') AS code, CHR(65) AS letter → code=65 letter=A

11 · Numeric functions

ROUND(x)	Rounds to the nearest whole number. SELECT ROUND(1234.5678) AS rounded → 1235
ROUND(x, n)	Rounds to n decimal places. Needs numeric — ROUND(double, n) does not exist in PostgreSQL. SELECT ROUND(1234.5678, 2) AS two_dp, ROUND(AVG(salary)::numeric, 2) AS avg_2dp FROM employees → two_dp=1234.57 avg_2dp=139687.50
CEIL(x) / CEILING(x)	Rounds UP to the next whole number. SELECT CEIL(4.1) AS up → 5
FLOOR(x)	Rounds DOWN. SELECT FLOOR(4.9) AS down → 4
TRUNC(x[,n])	Chops off decimals without rounding. SELECT TRUNC(4.99) AS chopped, TRUNC(4.99,1) AS one_dp → chopped=4 one_dp=4.9
ABS(x)	Absolute value. SELECT ABS(-17) AS positive → 17
MOD(a,b)	Remainder of a divided by b. SELECT MOD(17,5) AS remainder → 2
DIV(a,b)	Integer quotient, discarding the remainder. SELECT DIV(17,5) AS quotient → 3
POWER(a,b)	a to the power of b. SELECT POWER(2,10) AS result → 1024
SQRT(x) / CBRT(x)	Square root / cube root. SELECT SQRT(144) AS sq, CBRT(27) AS cb → sq=12 cb=3
EXP(x) / LN(x) / LOG(x)	e^x, natural log, base-10 log. SELECT ROUND(EXP(1)::numeric,4) AS e, LN(1) AS ln1, LOG(100) AS log100 → e=2.7183 ln1=0 log100=2

SIGN(x)	Returns -1, 0 or 1 depending on the sign. SELECT SIGN(-5) AS neg, SIGN(0) AS zero, SIGN(5) AS pos → neg=-1 zero=0 pos=1
RANDOM()	Random number in [0,1). Use with ORDER BY for a random sample. SELECT (RANDOM() >= 0 AND RANDOM() < 1) AS always_true → true
WIDTH_BUCKET(x, lo, hi, n)	Assigns a value to one of n equal-width buckets — histogram binning. SELECT WIDTH_BUCKET(85000, 80000, 260000, 4) AS bucket → 1

12 · Date & time functions

The single richest source of analyst interview questions.

CURRENT_DATE	Today, as a date. SELECT (CURRENT_DATE IS NOT NULL) AS works → true
NOW() / CURRENT_TIMESTAMP	This instant, with time zone. SELECT (NOW() IS NOT NULL) AS works → true
DATE_TRUNC(unit, ts)	Rounds a date DOWN to the start of the unit. The workhorse for monthly grouping. SELECT DATE_TRUNC('month', DATE '2024-05-15')::date AS month_start, DATE_TRUNC('week', DATE '2024-05-15')::date AS week_start → month_start=2024-05-01 week_start=2024-05-13
EXTRACT(field FROM ts)	Pulls one field out as a number: YEAR, MONTH, DAY, QUARTER, DOW, DOY, HOUR, EPOCH. SELECT EXTRACT(YEAR FROM DATE '2024-05-15') AS yr, EXTRACT(QUARTER FROM DATE '2024-05-15') AS qtr, EXTRACT(DOW FROM DATE '2024-05-15') AS dow → yr=2024 qtr=2 dow=3
DATE_PART(field, ts)	Function-call form of EXTRACT; identical result. SELECT DATE_PART('month', DATE '2024-05-15') AS mth → 5
TO_CHAR(ts, fmt)	Formats a date as TEXT for display. YYYY MM DD Mon Day HH24 MI SS. SELECT TO_CHAR(DATE '2024-05-15', 'YYYY-MM') AS label, TO_CHAR(DATE '2024-05-15', 'Mon DD, YYYY') AS pretty → label=2024-05 pretty=May 15, 2024
TO_DATE(s, fmt)	Parses text into a date using a format string. SELECT TO_DATE('15/05/2024', 'DD/MM/YYYY') AS parsed → 2024-05-15
TO_TIMESTAMP(s, fmt)	Parses text into a timestamp. SELECT TO_TIMESTAMP('2024-05-15 14:30', 'YYYY-MM-DD HH24:MI') AS parsed → 2024-05-15
MAKE_DATE(y,m,d)	Builds a date from three numbers. SELECT MAKE_DATE(2024,5,15) AS built → 2024-05-15
d + INTERVAL '1 month'	Date arithmetic. Units: year month week day hour minute second. SELECT DATE '2024-01-31' + INTERVAL '1 month' AS next_month, DATE '2024-05-15' - INTERVAL '7 days' AS week_ago → next_month=2024-02-29 week_ago=2024-05-08
date - date	Difference between two dates, in whole DAYS (an integer). SELECT DATE '2024-05-15' - DATE '2024-05-01' AS days_apart → 14

AGE(a, b)	Human-readable difference as an interval (years/months/days). SELECT AGE(DATE '2024-05-15', DATE '2023-01-10') AS gap → 1 year 4 mons 5 days
EXTRACT(EPOCH FROM x)	Converts an interval or timestamp to seconds — the way to get an exact numeric duration. SELECT EXTRACT(EPOCH FROM (TIMESTAMP '2024-05-15 12:00' - TIMESTAMP '2024-05-15 10:30'))/3600 AS hours → 1.5000000000000000
OVERLAPS	True if two date ranges overlap. SELECT (DATE '2024-01-01', DATE '2024-06-30') OVERLAPS (DATE '2024-06-01', DATE '2024-12-31') AS ranges_overlap → true
generate_series(d1,d2,step)	Builds a DATE SPINE so months with no data still appear as zero rows. SELECT g::date AS month FROM generate_series(DATE '2024-01-01', DATE '2024-06-01', INTERVAL '1 month') g → month=2024-01-01 · month=2024-02-01 ... 6 rows

13 · Aggregate functions

Many rows in, one value out. Every one of these ignores NULLs except COUNT(*).

COUNT(*)	Counts ROWS, including rows that are entirely NULL. SELECT COUNT(*) AS n FROM customers → 14
COUNT(col)	Counts NON-NULL values in that column. Almost always the source of a count mismatch. SELECT COUNT(*) AS all_rows, COUNT(city) AS with_city FROM customers → all_rows=14 with_city=12
COUNT(DISTINCT col)	Counts distinct non-NULL values. The fix for fan-out double counting. SELECT COUNT(DISTINCT country) AS countries FROM customers → 5
SUM(col)	Total. Skips NULLs; returns NULL (not 0) if every value is NULL. SELECT SUM(salary) AS payroll FROM employees → 2235000.00
AVG(col)	Mean. Skips NULLs in BOTH the total and the divisor — so it may not be the average the business means. SELECT ROUND(AVG(salary),0) AS mean_salary FROM employees → 139688
MIN(col) / MAX(col)	Extremes. Work on numbers, text and dates alike. SELECT MIN(salary) AS lowest, MAX(salary) AS highest FROM employees → lowest=85000.00 highest=250000.00
STRING_AGG(col, sep)	Glues values into one delimited string. Accepts its own ORDER BY. SELECT STRING_AGG(dept_name, ' ' ORDER BY dept_name) AS all_depts FROM departments → Engineering, Finance, Marketing, People Ops, Sales
ARRAY_AGG(col)	Collects values into an array. Unlike STRING_AGG it keeps NULLs. SELECT ARRAY_AGG(dept_id ORDER BY dept_id) AS ids FROM departments → [1,2,3,4,5]
BOOL_AND / BOOL_OR	True if all / any values are true. EVERY() is a synonym for BOOL_AND. SELECT BOOL_AND(salary>50000) AS all_above, BOOL_OR(salary>200000) AS any_above FROM employees → all_above=true any_above=true

STDDEV / VARIANCE	Sample standard deviation and variance. <code>_POP</code> variants give the population figures. <pre>SELECT ROUND(STDDEV(salary),0) AS sd, ROUND(VARIANCE(salary),0) AS var FROM employees</pre> → sd=43987 var=1934895833
PERCENTILE_CONT(f) WITHIN GROUP (ORDER BY x)	MEDIAN at <code>f=0.5</code> — interpolates between the two middle values. <pre>SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median FROM employees</pre> → 130000
PERCENTILE_DISC(f) WITHIN GROUP (ORDER BY x)	Percentile that returns an ACTUAL value from the data, never an interpolation. <pre>SELECT PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY salary) AS median_actual FROM employees</pre> → 130000.00
MODE() WITHIN GROUP (ORDER BY x)	The most frequently occurring value. <pre>SELECT MODE() WITHIN GROUP (ORDER BY salary) AS most_common FROM employees</pre> → 95000.00
CORR(y,x)	Pearson correlation coefficient between two columns. <pre>SELECT ROUND(CORR(unit_price, cost)::numeric,4) AS correlation FROM products</pre> → 0.9971
agg(...) FILTER (WHERE cond)	Aggregates only the rows matching a condition. The clean way to pivot. PostgreSQL/standard only. <pre>SELECT COUNT(*) AS total, COUNT(*) FILTER (WHERE status='completed') AS completed FROM orders</pre> → total=48 completed=37
SUM(CASE WHEN c THEN 1 ELSE 0 END)	The portable equivalent of FILTER — works in MySQL and SQL Server too. <pre>SELECT SUM(CASE WHEN status='completed' THEN 1 ELSE 0 END) AS completed FROM orders</pre> → 37
agg(DISTINCT col)	Aggregates only distinct values. <pre>SELECT SUM(DISTINCT salary) AS distinct_payroll FROM employees</pre> → 1755000.00

14 · GROUP BY & HAVING

Folding rows into groups, then filtering the groups.

GROUP BY col	Folds rows into one row per distinct value. EVERY non-aggregated SELECT column must appear here. <pre>SELECT country, COUNT(*) AS n FROM customers GROUP BY country ORDER BY n DESC</pre> → country=India n=6 · country=USA n=3 ... 5 rows
GROUP BY a, b	One row per COMBINATION of values. <pre>SELECT country, segment, COUNT(*) AS n FROM customers GROUP BY country, segment ORDER BY country, segment</pre> → country=Germany segment=Mid-Market n=1 · country=Germany segment=SMB n=1 ... 11 rows
GROUP BY 1	Groups by the first SELECT column by position. Convenient; name it in production code. <pre>SELECT DATE_TRUNC('month',order_date)::date AS m, COUNT(*) AS n FROM orders GROUP BY 1 ORDER BY 1</pre> → m=2024-01-01 n=6 · m=2024-02-01 n=7 ... 6 rows

Aggregate with no GROUP BY	Collapses the WHOLE table to a single row. <pre>SELECT COUNT(*) AS n, ROUND(AVG(salary),0) AS avg FROM employees</pre> → n=16 avg=139688
THE RULE	Every column in SELECT is either inside an aggregate, or listed in GROUP BY. There is no third option. <pre>SELECT dept_id, COUNT(*) ... GROUP BY dept_id ✓</pre> <pre>SELECT dept_id, emp_name, COUNT(*) ... GROUP BY dept_id x error</pre>
HAVING cond	Filters GROUPS after aggregation. The only place an aggregate may be used as a filter. <pre>SELECT country, COUNT(*) AS n FROM customers GROUP BY country HAVING COUNT(*) > 1</pre> → country=UK n=2 · country=USA n=3 ... 4 rows
WHERE vs HAVING	WHERE filters rows before grouping; HAVING filters groups after. No aggregate in the condition → it belongs in WHERE. <pre>SELECT cust_id, COUNT(*) AS n FROM orders WHERE status='completed' GROUP BY cust_id HAVING COUNT(*)>=3 ORDER BY n DESC</pre> → cust_id=1 n=7 · cust_id=2 n=5 ... 7 rows
GROUP BY ROLLUP(a,b)	Adds subtotal rows and a grand total — a hierarchy of totals in one pass. <pre>SELECT country, COUNT(*) AS n, GROUPING(country) AS is_total FROM customers GROUP BY ROLLUP(country) ORDER BY is_total, country</pre> → country=Germany n=2 is_total=0 · country=India n=6 is_total=0 ... 6 rows
GROUP BY CUBE(a,b)	Every possible combination of subtotals. <pre>SELECT country, segment, COUNT(*) AS n FROM customers GROUP BY CUBE(country, segment) ORDER BY country NULLS LAST, segment NULLS LAST</pre> → country=Germany segment=Mid-Market n=1 · country=Germany segment=SMB n=1 ... 20 rows
GROUP BY GROUPING SETS (...)	You list exactly which grouping levels you want. <pre>SELECT country, segment, COUNT(*) AS n FROM customers GROUP BY GROUPING SETS ((country),(segment),()) ORDER BY country NULLS LAST, segment NULLS LAST</pre> → country=Germany segment=NULL n=2 · country=India segment=NULL n=6 ... 9 rows
GROUPING(col)	Returns 1 when the row is a subtotal for that column — the only way to tell a subtotal NULL from a real NULL. <pre>SELECT country, GROUPING(country) AS is_subtotal, COUNT(*) AS n FROM customers GROUP BY ROLLUP(country) ORDER BY is_subtotal DESC LIMIT 3</pre> → country=NULL is_subtotal=1 n=14 · country=Singapore is_subtotal=0 n=1 ... 3 rows

15 · ORDER BY, LIMIT & OFFSET

ORDER BY col	Sorts the final rows. Ascending by default. Without it there is NO guaranteed order. <pre>SELECT emp_name, salary FROM employees ORDER BY salary DESC LIMIT 3</pre> → emp_name=Rohan Mehta salary=250000.00 · emp_name=Priya Nair salary=190000.00 ... 3 rows
ASC / DESC	Sort direction, per column. <pre>SELECT cust_name, country FROM customers ORDER BY country ASC, cust_name DESC LIMIT 4</pre> → cust_name=Solstice Labs country=Germany · cust_name=Falcon Motors country=Germany ... 4 rows
NULLS FIRST / NULLS LAST	Where NULLs go. PostgreSQL default: last when ASC, first when DESC. <pre>SELECT cust_name, city FROM customers ORDER BY city NULLS LAST LIMIT 4</pre> → cust_name=Zenith Retail city=Bengaluru · cust_name=Falcon Motors city=Berlin ... 4 rows

ORDER BY alias	Allowed, because ORDER BY runs AFTER SELECT. <pre>SELECT emp_name, salary*12 AS annual FROM employees ORDER BY annual DESC LIMIT 3</pre> → emp_name=Rohan Mehta annual=3000000.00 · emp_name=Priya Nair annual=2280000.00 ... 3 rows
ORDER BY 2	Sorts by the second SELECT column, by position. <pre>SELECT country, COUNT(*) AS n FROM customers GROUP BY country ORDER BY 2 DESC LIMIT 3</pre> → country=India n=6 · country=USA n=3 ... 3 rows
ORDER BY CASE ...	Custom, non-alphabetical ordering — e.g. a business status sequence. <pre>SELECT status, COUNT(*) AS n FROM orders GROUP BY status ORDER BY CASE status WHEN 'pending' THEN 1 WHEN 'completed' THEN 2 WHEN 'refunded' THEN 3 ELSE 4 END</pre> → status=pending n=4 · status=completed n=37 ... 4 rows
LIMIT n	Return at most n rows. <pre>SELECT emp_name FROM employees ORDER BY emp_id LIMIT 3</pre> → emp_name=Rohan Mehta · emp_name=Priya Nair ... 3 rows
OFFSET m	Skip the first m rows. Gets slower the deeper you page. <pre>SELECT emp_name FROM employees ORDER BY emp_id LIMIT 3 OFFSET 3</pre> → emp_name=Sneha Iyer · emp_name=Karthik Menon ... 3 rows
FETCH FIRST n ROWS ONLY	The ANSI spelling of LIMIT. PostgreSQL accepts both. <pre>SELECT emp_name FROM employees ORDER BY emp_id FETCH FIRST 2 ROWS ONLY</pre> → emp_name=Rohan Mehta · emp_name=Priya Nair (2 rows)
Stable paging	Always add a UNIQUE tie-break column to ORDER BY, or pages can repeat or skip rows. <pre>SELECT emp_name, salary FROM employees ORDER BY salary DESC, emp_id LIMIT 3 OFFSET 2</pre> → emp_name=Arjun Rao salary=185000.00 · emp_name=Sneha Iyer salary=175000.00 ... 3 rows
Keyset pagination	Replaces OFFSET with a row-value comparison. Constant speed at any depth. <pre>SELECT emp_name, salary FROM employees WHERE (salary, emp_id) < (150000, 6) ORDER BY salary DESC, emp_id DESC LIMIT 3</pre> → emp_name=Karthik Menon salary=150000.00 · emp_name=Meera Pillai salary=130000.00 ... 3 rows

16 · JOINS

Combining tables side by side. Mentally: every possible pairing, keeping the pairs where ON is true.

INNER JOIN ... ON	Keeps only rows that match on BOTH sides. The word INNER is optional. <pre>SELECT COUNT(*) AS n FROM customers c INNER JOIN orders o ON o.cust_id=c.cust_id</pre> → 48
LEFT JOIN ... ON	Keeps ALL left rows; right-side columns become NULL where there is no match. <pre>SELECT COUNT(*) AS n FROM customers c LEFT JOIN orders o ON o.cust_id=c.cust_id</pre> → 51
RIGHT JOIN ... ON	Keeps all RIGHT rows. Rare — flip the tables and write LEFT instead. <pre>SELECT COUNT(*) AS n FROM orders o RIGHT JOIN customers c ON o.cust_id=c.cust_id</pre> → 51
FULL OUTER JOIN ... ON	Keeps everything from both sides. Used to reconcile two systems. <pre>SELECT COUNT(*) AS n FROM customers c FULL OUTER JOIN orders o ON o.cust_id=c.cust_id</pre> → 51

CROSS JOIN	Every combination — the Cartesian product. No ON clause. Used for date spines and grids. <pre>SELECT COUNT(*) AS n FROM (SELECT 1 UNION SELECT 2) a CROSS JOIN (SELECT 1 UNION SELECT 2 UNION SELECT 3) b</pre> → 6
USING (col)	Shorthand for ON a.col = b.col when the column is named identically. Merges it into one output column. <pre>SELECT COUNT(*) AS n FROM orders JOIN customers USING (cust_id)</pre> → 48
NATURAL JOIN	Joins on EVERY same-named column. Avoid it — adding a column later silently changes the query. <pre>FROM a NATURAL JOIN b -- do not use</pre>
ON vs WHERE (outer join)	On a LEFT JOIN, a condition on the right table in WHERE deletes the NULL rows and silently makes it an INNER join. Put it in ON. <pre>SELECT (SELECT COUNT(*) FROM customers c LEFT JOIN orders o ON o.cust_id=c.cust_id AND o.status='completed') AS in_on, (SELECT COUNT(*) FROM customers c LEFT JOIN orders o ON o.cust_id=c.cust_id WHERE o.status='completed') AS in_where</pre> → in_on=40 in_where=37
Self join	A table joined to itself with two aliases — for hierarchies and row-to-row comparisons. <pre>SELECT e.emp_name AS employee, m.emp_name AS manager FROM employees e LEFT JOIN employees m ON m.emp_id=e.manager_id ORDER BY e.emp_id LIMIT 4</pre> → employee=Rohan Mehta manager=NULL · employee=Priya Nair manager=Rohan Mehta ... 4 rows
Anti-join	Rows in A with NO match in B. Write it as NOT EXISTS, or LEFT JOIN ... WHERE right IS NULL. <pre>SELECT COUNT(*) AS never_ordered FROM products p WHERE NOT EXISTS (SELECT 1 FROM order_items i WHERE i.product_id=p.product_id)</pre> → 2
Semi-join	Rows in A that have AT LEAST ONE match in B, without duplicating A. Write it as EXISTS. <pre>SELECT COUNT(*) AS ordered_at_least_once FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.cust_id=c.cust_id)</pre> → 11
Non-equi join	ON accepts any boolean, not just equality — used for banding and effective-dated records. <pre>WITH b(label,lo,hi) AS (VALUES ('mid',100000,129999), ('senior',130000,174999)) SELECT e.emp_name, b.label FROM employees e JOIN b ON e.salary BETWEEN b.lo AND b.hi ORDER BY e.salary LIMIT 4</pre> → emp_name=Rahul Verma label=mid · emp_name=Vikram Desai label=mid ... 4 rows
Fan-out	Joining a parent to a child MULTIPLIES the parent rows. Any SUM of a parent column is then double counted. <pre>SELECT COUNT(*) AS rows_after_join, COUNT(DISTINCT o.order_id) AS actual_orders FROM orders o JOIN order_items oi ON oi.order_id=o.order_id</pre> → rows_after_join=119 actual_orders=48
Aggregate-then-join	When two child tables hang off one parent, aggregate each separately BEFORE joining, or the rows multiply together. <pre>SELECT c.cust_name, COALESCE(i.rev,0) AS revenue FROM customers c LEFT JOIN (SELECT o.cust_id, SUM(oi.line_total) AS rev FROM orders o JOIN order_items oi ON oi.order_id=o.order_id GROUP BY o.cust_id) i ON i.cust_id=c.cust_id ORDER BY revenue DESC LIMIT 3</pre> → cust_name=Zenith Retail revenue=108195.00 · cust_name=Nimbus Cloud revenue=53662.50 ... 3 rows

17 · Set operations

Stacking results on top of each other. Same column count, compatible types; names come from the first query.

UNION	All rows from both queries, DUPLICATES REMOVED (which costs a sort or hash). SELECT COUNT (*) AS n FROM (SELECT country FROM customers UNION SELECT country FROM customers) t → 5
UNION ALL	All rows from both, duplicates KEPT. Faster — make it your default. SELECT COUNT (*) AS n FROM (SELECT country FROM customers UNION ALL SELECT country FROM customers) t → 28
INTERSECT	Only rows appearing in BOTH queries. SELECT city FROM customers WHERE city IS NOT NULL INTERSECT SELECT location FROM departments → city=Mumbai · city=Delhi ... 3 rows
EXCEPT	Rows in the first query that are NOT in the second. (MySQL/Oracle call it MINUS.) SELECT city FROM customers WHERE city IS NOT NULL EXCEPT SELECT location FROM departments ORDER BY city → city=Berlin · city=Boston ... 9 rows
INTERSECT ALL / EXCEPT ALL	Same, but keeping duplicate multiplicity. SELECT 1 UNION ALL SELECT 1 INTERSECT ALL SELECT 1 → ?column?=1 · ?column?=1 (2 rows)
NULL handling	Set operations compare WHOLE ROWS and treat NULL as equal to NULL — unlike the = operator. SELECT COUNT (*) AS n FROM (SELECT NULL:: int UNION SELECT NULL:: int) t → 1
ORDER BY placement	One ORDER BY, at the very end; it applies to the combined result. SELECT cust_name AS name FROM customers WHERE country='UK' UNION ALL SELECT emp_name FROM employees WHERE dept_id=3 ORDER BY name → name=Alder Consulting · name=Ishaan Bose ... 5 rows

18 · Subqueries

A query inside a query. Where it sits determines what it must return.

Scalar subquery in SELECT	Must return exactly ONE row and ONE column. Repeated on every output row. SELECT emp_name, salary, (SELECT ROUND (AVG (salary)) FROM employees) AS company_avg FROM employees LIMIT 3 → emp_name=Rohan Mehta salary=250000.00 company_avg=139688 · emp_name=Priya Nair salary=190000.00 company_avg=139688 ... 3 rows
Scalar subquery in WHERE	Compares each row against a single computed value. SELECT COUNT (*) AS above_avg FROM employees WHERE salary > (SELECT AVG (salary) FROM employees) → 7
Derived table in FROM	A subquery used as a table. The alias is mandatory. SELECT AVG (t.n) AS avg_per_dept FROM (SELECT dept_id, COUNT (*) AS n FROM employees GROUP BY dept_id) t → 4.0000000000000000
IN (subquery)	Membership test against one column of any number of rows. SELECT COUNT (*) AS n FROM orders WHERE cust_id IN (SELECT cust_id FROM customers WHERE country='India') → 25

Correlated subquery	References a column from the OUTER query, so conceptually it re-runs for every outer row. <pre>SELECT COUNT(*) AS n FROM employees e WHERE e.salary > (SELECT AVG(e2.salary) FROM employees e2 WHERE e2.dept_id=e.dept_id)</pre> → 6
Correlated scalar in SELECT	Adds a per-row lookup — e.g. each customer with their own latest order date. <pre>SELECT c.cust_name, (SELECT MAX(o.order_date) FROM orders o WHERE o.cust_id=c.cust_id) AS last_order FROM customers c ORDER BY c.cust_id LIMIT 4</pre> → cust_name=Zenith Retail last_order=2024-06-15 · cust_name=Lumen Analytics last_order=2024-06-03 ... 4 rows
LATERAL (subquery)	Lets a FROM subquery see columns from the item before it. The clean top-N-per-group. SQL Server calls this CROSS APPLY. <pre>SELECT d.dept_name, t.emp_name FROM departments d CROSS JOIN LATERAL (SELECT emp_name FROM employees e WHERE e.dept_id=d.dept_id ORDER BY salary DESC LIMIT 1) t</pre> → dept_name=Engineering emp_name=Priya Nair · dept_name=Sales emp_name=Arjun Rao ... 3 rows
EXISTS vs IN vs JOIN	EXISTS for "has any" (never duplicates rows). NOT EXISTS for "has none" (NULL-safe). JOIN when you need the other table columns. NOT IN + a NULL in the list → zero rows. Always prefer NOT EXISTS.

19 · CTEs (WITH clause)

Named intermediate results. The biggest readability upgrade available to you.

WITH name AS (...)	Defines a named temporary result used by the query below it. <pre>WITH big AS (SELECT * FROM employees WHERE salary>130000) SELECT COUNT(*) AS n FROM big</pre> → 7
Multiple CTEs	Comma-separated; each may use any CTE defined above it. <pre>WITH a AS (SELECT * FROM orders WHERE status='completed'), b AS (SELECT cust_id, COUNT(*) AS n FROM a GROUP BY cust_id) SELECT COUNT(*) AS customers FROM b</pre> → 11
WITH RECURSIVE	Anchor term, UNION ALL, then a recursive term that references the CTE. For hierarchies and series. <pre>WITH RECURSIVE n(x) AS (SELECT 1 UNION ALL SELECT x+1 FROM n WHERE x<5) SELECT STRING_AGG(x::text, ',') AS series FROM n</pre> → 1,2,3,4,5
Recursive hierarchy	Walk an org chart, tracking depth and path. <pre>WITH RECURSIVE org AS (SELECT emp_id, emp_name, 1 AS lvl FROM employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.emp_name, o.lvl+1 FROM employees e JOIN org o ON e.manager_id=o.emp_id) SELECT lvl, COUNT(*) AS people FROM org GROUP BY lvl ORDER BY lvl</pre> → lvl=1 people=1 · lvl=2 people=3 ... 4 rows
Stop condition	The recursive term MUST eventually return no rows, or the query never ends. <pre>... UNION ALL SELECT x+1 FROM n WHERE x < 100 -- the WHERE is the brake</pre>
MATERIALIZED / NOT MATERIALIZED	Forces the planner to compute a CTE once, or to inline it. Default since PG12 is to inline when safe. <pre>WITH x AS MATERIALIZED (SELECT 1 AS a) SELECT * FROM x</pre> → 1
Data-modifying CTE	INSERT/UPDATE/DELETE ... RETURNING can be a CTE — how you move rows between tables atomically. <pre>WITH moved AS (DELETE FROM a WHERE ... RETURNING *) INSERT INTO b SELECT * FROM moved;</pre>

20 · Window functions

Aggregate *WITHOUT* collapsing rows. The highest-value topic for an analyst interview.

fn() OVER ()	Computes over ALL rows, adding a whole-table value to every row. <pre>SELECT emp_name, salary, SUM(salary) OVER () AS total_payroll FROM employees LIMIT 3</pre> → emp_name=Rohan Mehta salary=250000.00 total_payroll=2235000.00 · emp_name=Priya Nair salary=190000.00 total_payroll=2235000.00 ... 3 rows
OVER (PARTITION BY col)	Splits rows into independent groups; the function restarts for each. <pre>SELECT emp_name, dept_id, ROUND(AVG(salary) OVER (PARTITION BY dept_id)) AS dept_avg FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id LIMIT 4</pre> → emp_name=Priya Nair dept_id=1 dept_avg=139375 · emp_name=Karthik Menon dept_id=1 dept_avg=139375 ... 4 rows
OVER (ORDER BY col)	Orders rows within the partition — and SILENTLY sets a running frame. <pre>SELECT order_id, SUM(1) OVER (ORDER BY order_id) AS running_count FROM orders LIMIT 4</pre> → order_id=1000 running_count=1 · order_id=1001 running_count=2 ... 4 rows
Default frame	With ORDER BY the frame becomes RANGE UNBOUNDED PRECEDING TO CURRENT ROW, so aggregates become RUNNING totals. Without ORDER BY it is the whole partition. SUM(x) OVER (ORDER BY d) = running total, NOT a grand total
ROWS BETWEEN a AND b	Frame counted in PHYSICAL ROWS. Usually what you actually mean. <pre>SELECT order_id, ROUND(AVG(order_id) OVER (ORDER BY order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW),1) AS moving_avg FROM orders LIMIT 5</pre> → order_id=1000 moving_avg=1000.0 · order_id=1001 moving_avg=1000.5 ... 5 rows
RANGE BETWEEN a AND b	Frame counted in VALUES — all rows sharing the ORDER BY value (peers) are included together. <pre>SELECT order_id, SUM(1) OVER (ORDER BY order_id RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS n FROM orders LIMIT 4</pre> → order_id=1000 n=1 · order_id=1001 n=2 ... 4 rows
GROUPS BETWEEN a AND b	Frame counted in peer GROUPS rather than rows or values. <pre>SELECT order_id, SUM(1) OVER (ORDER BY order_id GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW) AS n FROM orders LIMIT 4</pre> → order_id=1000 n=1 · order_id=1001 n=2 ... 4 rows
UNBOUNDED PRECEDING / FOLLOWING	The first / last row of the partition. <pre>SELECT order_id, SUM(1) OVER (ORDER BY order_id ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS all_rows FROM orders LIMIT 3</pre> → order_id=1000 all_rows=48 · order_id=1001 all_rows=48 ... 3 rows
n PRECEDING / n FOLLOWING	n rows back / forward from the current row. <pre>SELECT order_id, SUM(1) OVER (ORDER BY order_id ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS window_size FROM orders LIMIT 4</pre> → order_id=1000 window_size=2 · order_id=1001 window_size=3 ... 4 rows
EXCLUDE CURRENT ROW / TIES / GROUP	Removes rows from the frame — e.g. an average of everyone EXCEPT me. <pre>SELECT order_id, SUM(1) OVER (ORDER BY order_id ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING EXCLUDE CURRENT ROW) AS others FROM orders LIMIT 3</pre> → order_id=1000 others=47 · order_id=1001 others=47 ... 3 rows
ROW_NUMBER()	1,2,3... always distinct, arbitrary on ties. Use for de-duplication and exact top-N. <pre>SELECT emp_name, salary, ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn FROM employees LIMIT 5</pre> → emp_name=Rohan Mehta salary=250000.00 rn=1 · emp_name=Priya Nair salary=190000.00 rn=2 ... 5 rows

RANK()	Ties share a rank and leave a GAP: 1,1,3. <pre>SELECT emp_name, salary, RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees LIMIT 5</pre> → emp_name=Rohan Mehta salary=250000.00 rnk=1 · emp_name=Priya Nair salary=190000.00 rnk=2 ... 5 rows
DENSE_RANK()	Ties share a rank with NO gap: 1,1,2. Use for "top 3 distinct salaries". <pre>SELECT emp_name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk FROM employees LIMIT 5</pre> → emp_name=Rohan Mehta salary=250000.00 drnk=1 · emp_name=Priya Nair salary=190000.00 drnk=2 ... 5 rows
PERCENT_RANK()	Relative rank from 0 to 1: (rank – 1) / (rows – 1). <pre>SELECT emp_name, ROUND(PERCENT_RANK() OVER (ORDER BY salary)::numeric,3) AS pct FROM employees LIMIT 4</pre> → emp_name=Tara Krishnan pct=0.000 · emp_name=Sanjay Joshi pct=0.067 ... 4 rows
CUME_DIST()	Cumulative distribution — the fraction of rows at or below this one. <pre>SELECT emp_name, ROUND(CUME_DIST() OVER (ORDER BY salary)::numeric,3) AS cd FROM employees LIMIT 4</pre> → emp_name=Tara Krishnan cd=0.063 · emp_name=Sanjay Joshi cd=0.188 ... 4 rows
NTILE(n)	Splits rows into n roughly equal buckets — quartiles, deciles. <pre>SELECT emp_name, salary, NTILE(4) OVER (ORDER BY salary) AS quartile FROM employees LIMIT 5</pre> → emp_name=Tara Krishnan salary=85000.00 quartile=1 · emp_name=Sanjay Joshi salary=95000.00 quartile=1 ... 5 rows
LAG(col[,offset[,default]])	Value from a PREVIOUS row. The basis of period-over-period change. <pre>SELECT order_id, LAG(order_id) OVER (ORDER BY order_id) AS prev FROM orders LIMIT 4</pre> → order_id=1000 prev=NULL · order_id=1001 prev=1000 ... 4 rows
LEAD(col[,offset[,default]])	Value from a FOLLOWING row. <pre>SELECT order_id, LEAD(order_id) OVER (ORDER BY order_id) AS next FROM orders LIMIT 4</pre> → order_id=1000 next=1001 · order_id=1001 next=1002 ... 4 rows
FIRST_VALUE(col)	First value in the frame. Correct with the default frame. <pre>SELECT dept_id, emp_name, FIRST_VALUE(emp_name) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS top_earner FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id LIMIT 4</pre> → dept_id=1 emp_name=Priya Nair top_earner=Priya Nair · dept_id=1 emp_name=Ananya Gupta top_earner=Priya Nair ... 4 rows
LAST_VALUE(col)	Last value in the frame — returns the CURRENT row unless you widen the frame explicitly. <pre>SELECT dept_id, emp_name, LAST_VALUE(emp_name) OVER (PARTITION BY dept_id ORDER BY salary DESC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS lowest_paid FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id LIMIT 4</pre> → dept_id=1 emp_name=Priya Nair lowest_paid=Rahul Verma · dept_id=1 emp_name=Ananya Gupta lowest_paid=Rahul Verma ... 4 rows
NTH_VALUE(col, n)	The nth value in the frame. <pre>SELECT dept_id, NTH_VALUE(emp_name,2) OVER (PARTITION BY dept_id ORDER BY salary DESC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS second FROM employees WHERE dept_id IS NOT NULL LIMIT 4</pre> → dept_id=1 second=Ananya Gupta · dept_id=1 second=Ananya Gupta ... 4 rows
SUM/AVG/COUNT/MIN/MAX OVER	Any aggregate can be a window function — running totals, moving averages, share of total. <pre>SELECT emp_name, salary, ROUND(100.0*salary/SUM(salary) OVER (),2) AS pct_of_payroll FROM employees LIMIT 4</pre> → emp_name=Rohan Mehta salary=250000.00 pct_of_payroll=11.19 · emp_name=Priya Nair salary=190000.00 pct_of_payroll=8.50 ... 4 rows

WINDOW w AS (...)	Names a window spec so several functions can reuse it. Sits between HAVING and ORDER BY. <pre>SELECT emp_name, RANK() OVER w AS rnk, DENSE_RANK() OVER w AS drnk FROM employees WINDOW w AS (ORDER BY salary DESC) LIMIT 4</pre> → emp_name=Rohan Mehta rnk=1 drnk=1 · emp_name=Priya Nair rnk=2 drnk=2 ... 4 rows
Not allowed in WHERE/GROUP BY/HAVING	Windows are computed at step 6, after those clauses. Wrap the query in a CTE and filter outside. <pre>WITH r AS (SELECT emp_name, dept_id, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rk FROM employees WHERE dept_id IS NOT NULL) SELECT * FROM r WHERE rk<=2 ORDER BY dept_id, rk</pre> → emp_name=Priya Nair dept_id=1 rk=1 · emp_name=Ananya Gupta dept_id=1 rk=2 ... 6 rows
Window over an aggregate	Windows run AFTER GROUP BY, so you can rank grouped rows. SUM(SUM(x)) OVER () is the grand total of the groups. <pre>SELECT country, COUNT(*) AS n, ROUND(100.0*COUNT(*)/SUM(COUNT(*)) OVER (),1) AS pct FROM customers GROUP BY country ORDER BY n DESC</pre> → country=India n=6 pct=42.9 · country=USA n=3 pct=21.4 ... 5 rows

21 · DDL — defining tables

Data Definition Language: the SHAPE of the data. Examples build demo_ tables so nothing else is disturbed.

CREATE TABLE t (...)	Creates a table. Commas BETWEEN column definitions, none after the last. <pre>DROP TABLE IF EXISTS demo_t; CREATE TABLE demo_t (id integer, name text); SELECT 'created' AS status</pre> → created
IF NOT EXISTS	Makes creation idempotent — no error if it is already there. <pre>CREATE TABLE IF NOT EXISTS demo_t (id integer); SELECT 'ok' AS status</pre> → ok
PRIMARY KEY	The row identity. Implies UNIQUE + NOT NULL. One per table. <pre>DROP TABLE IF EXISTS demo_pk; CREATE TABLE demo_pk (id integer PRIMARY KEY, v text); SELECT 'ok' AS status</pre> → ok
PRIMARY KEY (a, b)	Composite key across several columns — only expressible at table level. <pre>DROP TABLE IF EXISTS demo_ck; CREATE TABLE demo_ck (a integer, b integer, PRIMARY KEY (a,b)); SELECT 'ok' AS status</pre> → ok
UNIQUE	No two rows share a value — but MANY NULLs are still allowed, because NULL <> NULL. <pre>DROP TABLE IF EXISTS demo_u; CREATE TABLE demo_u (code text UNIQUE); INSERT INTO demo_u VALUES (NULL),(NULL),(NULL); SELECT COUNT(*) AS nulls_allowed FROM demo_u</pre> → 3
NOT NULL	A value must be supplied. <pre>DROP TABLE IF EXISTS demo_nn; CREATE TABLE demo_nn (a integer NOT NULL); SELECT 'ok' AS status</pre> → ok
DEFAULT value	Value used when none is given on insert. <pre>DROP TABLE IF EXISTS demo_d; CREATE TABLE demo_d (a integer, country text DEFAULT 'India'); INSERT INTO demo_d (a) VALUES (1); SELECT * FROM demo_d</pre> → a=1 country=India
CHECK (condition)	An arbitrary per-row rule. Passes if it evaluates to NULL. <pre>DROP TABLE IF EXISTS demo_c; CREATE TABLE demo_c (age integer CHECK (age >= 18)); SELECT 'ok' AS status</pre> → ok

CONSTRAINT name CHECK (...)	A NAMED constraint — so the error message and any later DROP can refer to it. <pre>DROP TABLE IF EXISTS demo_nc; CREATE TABLE demo_nc (p numeric, c numeric, CONSTRAINT price_above_cost CHECK (p > c)); SELECT 'ok' AS status</pre> → ok
REFERENCES t(col)	Foreign key — the value must exist in the parent table. <pre>DROP TABLE IF EXISTS demo_child; CREATE TABLE demo_child (id integer, pk_id integer REFERENCES demo_pk(id)); SELECT 'ok' AS status</pre> → ok
ON DELETE CASCADE	Deleting the parent deletes the children. For rows with no meaning alone, e.g. order_items. <pre>order_id integer REFERENCES orders(order_id) ON DELETE CASCADE</pre>
ON DELETE RESTRICT / NO ACTION	Refuses to delete a parent that still has children. RESTRICT checks immediately. <pre>product_id integer REFERENCES products(product_id) ON DELETE RESTRICT</pre>
ON DELETE SET NULL / SET DEFAULT	Blanks the child FK instead of deleting the child row. <pre>dept_id integer REFERENCES departments(dept_id) ON DELETE SET NULL</pre>
GENERATED ALWAYS AS (expr) STORED	A column the DATABASE computes; you never write to it. <pre>DROP TABLE IF EXISTS demo_g; CREATE TABLE demo_g (q integer, p numeric, total numeric GENERATED ALWAYS AS (q*p) STORED); INSERT INTO demo_g (q,p) VALUES (3,10); SELECT * FROM demo_g</pre> → q=3 p=10 total=30
ALTER TABLE ... ADD COLUMN	Adds a column. <pre>ALTER TABLE demo_t ADD COLUMN city text; SELECT 'ok' AS status</pre> → ok
ALTER TABLE ... DROP COLUMN	Removes a column and its data. <pre>ALTER TABLE demo_t DROP COLUMN city; SELECT 'ok' AS status</pre> → ok
ALTER TABLE ... RENAME COLUMN	Renames a column. <pre>ALTER TABLE demo_t RENAME COLUMN name TO full_name; SELECT 'ok' AS status</pre> → ok
ALTER TABLE ... RENAME TO	Renames the table. <pre>ALTER TABLE demo_t RENAME TO demo_t2; ALTER TABLE demo_t2 RENAME TO demo_t; SELECT 'ok' AS status</pre> → ok
ALTER TABLE ... ALTER COLUMN TYPE	Changes a column type; USING supplies the conversion when it is not automatic. <pre>ALTER TABLE demo_t ALTER COLUMN id TYPE bigint; SELECT 'ok' AS status</pre> → ok
ALTER TABLE ... SET/DROP NOT NULL	Adds or removes the NOT NULL rule. <pre>ALTER TABLE demo_t ALTER COLUMN id SET NOT NULL; SELECT 'ok' AS status</pre> → ok
ALTER TABLE ... ADD/DROP CONSTRAINT	Adds or removes a named constraint after the fact. <pre>ALTER TABLE demo_t ADD CONSTRAINT id_positive CHECK (id > 0); ALTER TABLE demo_t DROP CONSTRAINT id_positive; SELECT 'ok' AS status</pre> → ok
DROP TABLE [IF EXISTS] t [CASCADE]	Removes the TABLE ITSELF. CASCADE also drops dependent objects. <pre>DROP TABLE IF EXISTS demo_gone; SELECT 'ok' AS status</pre> → ok

TRUNCATE TABLE t	Removes ALL rows fast, keeping the table. No WHERE, no triggers. In PostgreSQL it IS transactional. <pre>DROP TABLE IF EXISTS demo_tr; CREATE TABLE demo_tr (a int); INSERT INTO demo_tr VALUES (1),(2); TRUNCATE TABLE demo_tr; SELECT COUNT(*) AS rows_left FROM demo_tr</pre> → 0
CREATE TABLE ... AS SELECT	"CTAS" — builds a new table from a query result. <pre>DROP TABLE IF EXISTS demo_ctas; CREATE TABLE demo_ctas AS SELECT country, COUNT(*) AS n FROM customers GROUP BY country; SELECT COUNT(*) AS rows FROM demo_ctas</pre> → 5
CREATE TEMP TABLE	A table that disappears at the end of the session. <pre>CREATE TEMP TABLE demo_tmp AS SELECT 1 AS a; SELECT * FROM demo_tmp</pre> → 1
CREATE INDEX	Speeds up reads on a column, at the cost of slower writes. <pre>CREATE INDEX IF NOT EXISTS idx_demo ON demo_ctas (country); SELECT 'ok' AS status</pre> → ok
CREATE UNIQUE INDEX	Index that also enforces uniqueness. <pre>DROP TABLE IF EXISTS demo_ui; CREATE TABLE demo_ui (a int); CREATE UNIQUE INDEX idx_ui ON demo_ui (a); SELECT 'ok' AS status</pre> → ok
CREATE INDEX ... (a, b)	Composite index. Serves filters on a, or a AND b — but NOT b alone (leftmost-prefix rule). <pre>CREATE INDEX IF NOT EXISTS idx_demo2 ON demo_ctas (country, n); SELECT 'ok' AS status</pre> → ok
CREATE INDEX ... WHERE cond	PARTIAL index — indexes only the rows you actually query. Smaller and faster. <pre>CREATE INDEX IF NOT EXISTS idx_demo3 ON demo_ctas (n) WHERE n > 1; SELECT 'ok' AS status</pre> → ok
CREATE INDEX ... (LOWER(col))	EXPRESSION index — makes a function-wrapped WHERE sargable again. <pre>CREATE INDEX IF NOT EXISTS idx_demo4 ON demo_ctas (LOWER(country)); SELECT 'ok' AS status</pre> → ok
DROP INDEX	Removes an index. <pre>DROP INDEX IF EXISTS idx_demo4; SELECT 'ok' AS status</pre> → ok

22 · DML — changing data

Data Manipulation Language: the ROWS, not the shape.

INSERT INTO t (cols) VALUES (...)	Adds one row. ALWAYS name the columns — positional inserts break when a column is added. <pre>DROP TABLE IF EXISTS demo_i; CREATE TABLE demo_i (id int, name text); INSERT INTO demo_i (id,name) VALUES (1,'Asha'); SELECT * FROM demo_i</pre> → id=1 name=Asha
INSERT ... VALUES (...), (...), (...)	Multi-row insert in ONE statement — far faster than many statements. <pre>INSERT INTO demo_i (id,name) VALUES (2,'Ben'),(3,'Cara'); SELECT COUNT(*) AS rows FROM demo_i</pre> → 3

INSERT ... SELECT	Inserts the result of a query. <pre>INSERT INTO demo_i (id,name) SELECT 10+cust_id, cust_name FROM customers LIMIT 2; SELECT COUNT(*) AS rows FROM demo_i</pre> → 5
RETURNING *	PostgreSQL: gives back the rows actually written, including generated defaults. <pre>INSERT INTO demo_i (id,name) VALUES (99,'Zed') RETURNING id, name</pre> → id=99 name=Zed
UPDATE t SET c = v WHERE ...	Changes existing rows. Commas BETWEEN assignments. Omit WHERE and you update EVERY row. <pre>UPDATE demo_i SET name='Updated' WHERE id=1 RETURNING id, name</pre> → id=1 name=Updated
UPDATE ... FROM other	PostgreSQL: update using values joined from another table. <pre>UPDATE demo_i d SET name=c.cust_name FROM customers c WHERE c.cust_id=d.id RETURNING d.id, d.name</pre> → id=2 name=Lumen Analytics · id=3 name=Orbit Logistics ... 5 rows
DELETE FROM t WHERE ...	Removes rows. Omit WHERE and the table is emptied. <pre>DELETE FROM demo_i WHERE id=99 RETURNING id</pre> → 99
DELETE ... USING other	PostgreSQL: delete based on a join to another table. <pre>DELETE FROM demo_i USING customers c WHERE c.cust_id=demo_i.id AND c.country='Nowhere'; SELECT COUNT(*) AS remaining FROM demo_i</pre> → 5
INSERT ... ON CONFLICT DO NOTHING	UPSERT: skip the row silently if it already exists. <pre>DROP TABLE IF EXISTS demo_up; CREATE TABLE demo_up (sku text PRIMARY KEY, qty int); INSERT INTO demo_up VALUES ('A1',10); INSERT INTO demo_up VALUES ('A1',99) ON CONFLICT (sku) DO NOTHING; SELECT * FROM demo_up</pre> → sku=A1 qty=10
INSERT ... ON CONFLICT DO UPDATE	UPSERT: update the existing row instead. EXCLUDED is the row you TRIED to insert. <pre>INSERT INTO demo_up (sku,qty) VALUES ('A1',4),('C3',7) ON CONFLICT (sku) DO UPDATE SET qty = demo_up.qty + EXCLUDED.qty RETURNING sku, qty</pre> → sku=A1 qty=14 · sku=C3 qty=7 (2 rows)
MERGE INTO ... USING ... WHEN	ANSI upsert (PG15+). The standard answer in SQL Server and Oracle. <pre>MERGE INTO demo_up t USING (VALUES ('B2',100)) AS s(sku,qty) ON t.sku=s.sku WHEN MATCHED THEN UPDATE SET qty=s.qty WHEN NOT MATCHED THEN INSERT (sku,qty) VALUES (s.sku,s.qty); SELECT COUNT(*) AS rows FROM demo_up</pre> → 3

23 · Transactions & concurrency

BEGIN ... COMMIT	Groups statements so they all succeed or all fail. ACID: Atomicity, Consistency, Isolation, Durability. <pre>BEGIN; INSERT INTO demo_up VALUES ('T1',1); COMMIT; SELECT COUNT(*) AS rows FROM demo_up</pre> → 4
ROLLBACK	Undoes everything since BEGIN. <pre>BEGIN; DELETE FROM demo_up; ROLLBACK; SELECT COUNT(*) AS still_there FROM demo_up</pre> → 4
SAVEPOINT / ROLLBACK TO	A partial undo point inside a transaction. <pre>BEGIN; SAVEPOINT sp1; DELETE FROM demo_up; ROLLBACK TO sp1; COMMIT; SELECT COUNT(*) AS rows FROM demo_up</pre> → 4

READ COMMITTED	PostgreSQL DEFAULT. No dirty reads; non-repeatable reads and phantoms still possible. BEGIN ISOLATION LEVEL READ COMMITTED;
REPEATABLE READ	MySQL default. Rows you re-read do not change. In PostgreSQL this also blocks phantoms. BEGIN ISOLATION LEVEL REPEATABLE READ;
SERIALIZABLE	Strictest: the result is as if transactions ran one after another. BEGIN ISOLATION LEVEL SERIALIZABLE;
Dirty read	Reading another transaction UNCOMMITTED change. Impossible in PostgreSQL at any level. prevented by READ COMMITTED and above
Non-repeatable read	Reading the same ROW twice and getting different values. prevented by REPEATABLE READ and above
Phantom read	Re-running a RANGE query and finding new rows. prevented by SERIALIZABLE (and by PG REPEATABLE READ)
Lost update	Two transactions read-modify-write; one silently overwrites the other. Prevent with SELECT ... FOR UPDATE. SELECT ... FOR UPDATE;
SELECT ... FOR UPDATE	Locks the selected rows so nobody else can change them until you commit. SELECT * FROM accounts WHERE id=1 FOR UPDATE;
FOR UPDATE SKIP LOCKED	Takes only rows nobody else holds — the queue-worker pattern. SELECT * FROM jobs WHERE state='new' LIMIT 1 FOR UPDATE SKIP LOCKED;
MVCC	PostgreSQL keeps multiple row versions, so READERS NEVER BLOCK WRITERS and writers never block readers. the one-line answer to "how does Postgres handle concurrency?"
Deadlock	Two transactions each hold what the other needs. PostgreSQL detects it and kills one. Prevent by always locking in the same order. keep transactions short; lock in a consistent order

24 · Views, functions & triggers

CREATE VIEW v AS SELECT ...	A SAVED QUERY. Stores no data, always current, as slow as the underlying query. CREATE OR REPLACE VIEW demo_v AS SELECT country, COUNT(*) AS n FROM customers GROUP BY country; SELECT * FROM demo_v ORDER BY n DESC LIMIT 3 → country=India n=6 · country=USA n=3 ... 3 rows
CREATE MATERIALIZED VIEW	STORES the result. Fast to read, can be indexed, but goes stale until refreshed. DROP MATERIALIZED VIEW IF EXISTS demo_mv; CREATE MATERIALIZED VIEW demo_mv AS SELECT country, COUNT(*) AS n FROM customers GROUP BY country; SELECT COUNT(*) AS rows FROM demo_mv → 5
REFRESH MATERIALIZED VIEW	Recomputes it. CONCURRENTLY avoids locking readers (needs a unique index). REFRESH MATERIALIZED VIEW demo_mv; SELECT 'refreshed' AS status → refreshed
CREATE FUNCTION ... RETURNS	Returns a value or a table, and CAN be used inside a query. CREATE OR REPLACE FUNCTION demo_double(x integer) RETURNS integer LANGUAGE sql IMMUTABLE AS \$\$ SELECT x*2 \$\$; SELECT demo_double(21) AS result → 42
CREATE PROCEDURE ... / CALL	Returns nothing and CANNOT be used inside a query — but it CAN commit internally. That is the reason to use one. CREATE PROCEDURE p() LANGUAGE plpgsql AS \$\$ BEGIN ... COMMIT; END \$\$; CALL p();

CREATE TRIGGER	Runs a function automatically on INSERT/UPDATE/DELETE. OLD and NEW are the row before and after. CREATE TRIGGER trg AFTER UPDATE ON t FOR EACH ROW EXECUTE FUNCTION f();
BEFORE vs AFTER trigger	BEFORE can modify NEW or cancel the write; AFTER cannot, and is used for auditing. BEFORE → validate/modify AFTER → log/cascade

25 · Performance & inspection

EXPLAIN query	Shows the PLAN and estimated cost without running the query. EXPLAIN SELECT * FROM orders WHERE status='completed' → QUERY PLAN=Seq Scan on orders (cost=0.00..19.38 rows=4 width=80) · QUERY PLAN= Filter: (status = 'completed'::text) (2 rows)
EXPLAIN ANALYZE query	Actually RUNS it and reports real times and real row counts. Always start here. EXPLAIN ANALYZE SELECT COUNT(*) FROM orders WHERE status='completed' → QUERY PLAN=Aggregate (cost=19.39..19.40 rows=1 width=8) (actual time=0.131..0.134 rows=1.00 loops=1) · QUERY PLAN= Buffers: shared hit=1 ... 8 rows
Seq Scan	Reads the whole table. Fine on small tables; a red flag on a big one with a selective filter. fix: add an index on the filtered column
Index Scan / Index Only Scan	Walks the index. "Index Only" means the answer came from the index alone — the best case. Index Only Scan is why you avoid SELECT *
Nested Loop / Hash Join / Merge Join	The three join strategies: probe per row / build a hash table / zip two sorted inputs. Nested Loop is bad when the outer side is large
Estimated vs actual rows	A big gap means the planner statistics are stale. Run ANALYZE on the table. rows=1000 ... actual rows=2000000 → ANALYZE tablename;
Sargable	A predicate an index can serve. Keep the COLUMN bare on one side; do the arithmetic on the other. EXTRACT(YEAR FROM d) =2024 ✗ d >= '2024-01-01' AND d < '2025-01-01' ✓
Leftmost-prefix rule	Index (a,b) serves filters on a, or a AND b — never b alone. equality columns first , range column last
information_schema.columns	Standard catalogue view: inspect the columns of any table. SELECT column_name, data_type FROM information_schema.columns WHERE table_name='customers' ORDER BY ordinal_position LIMIT 5 → column_name=cust_id data_type=integer · column_name=cust_name data_type=text ... 5 rows
information_schema.tables	Lists the tables in the database. SELECT COUNT(*) AS tables FROM information_schema.tables WHERE table_schema='public' AND table_type='BASE TABLE' → 23

26 · How NULL behaves, everywhere

One table to settle every NULL argument. NULL means UNKNOWN, not zero and not blank.

= NULL	NEVER true, not even NULL = NULL. Always use IS NULL. SELECT (NULL = NULL) AS never_true, (NULL IS NULL) AS correct_test → never_true=NULL correct_test=true
---------------	---

NULL in arithmetic	Any arithmetic involving NULL gives NULL. SELECT NULL + 10 AS sum, NULL * 0 AS product → sum=NULL product=NULL
NULL in concatenation	With one NULL wipes the whole string. CONCAT and CONCAT_WS do not. SELECT ('abc' NULL) AS pipes, CONCAT ('abc',NULL) AS concat_fn → pipes=NULL concat_fn=abc
NULL in COUNT	COUNT(*) counts the row; COUNT(col) skips it. SELECT COUNT(*) AS star, COUNT(city) AS col FROM customers → star=14 col=12
NULL in SUM / AVG	Skipped. SUM of an all-NULL group is NULL, not 0. AVG divides by the NON-NULL count. SELECT SUM(x) AS sum_all_null, AVG(x) AS avg_all_null FROM (SELECT NULL::int AS x) t → sum_all_null=NULL avg_all_null=NULL
NULL in GROUP BY	All NULLs collapse into ONE group. SELECT COUNT(*) AS null_city_group FROM customers WHERE city IS NULL → 2
NULL in ORDER BY	PostgreSQL sorts NULLs LAST ascending, FIRST descending. Override with NULLS FIRST/LAST. SELECT city FROM customers ORDER BY city DESC LIMIT 3 → city=NULL · city=NULL ... 3 rows
NULL in DISTINCT	All NULLs count as ONE value. SELECT COUNT(*) AS n FROM (SELECT DISTINCT city FROM customers) t → 13
NULL in a JOIN key	NULL never matches anything, not even another NULL — those rows simply drop out of an inner join. SELECT COUNT(*) AS employees_kept FROM employees e JOIN departments d ON d.dept_id=e.dept_id → 15
NULL in UNIQUE	A UNIQUE constraint permits MANY NULL rows. SELECT COUNT(*) AS n FROM demo_u → 3
NULL in CHECK	A CHECK constraint PASSES when its condition evaluates to NULL. CHECK (age >= 18) accepts a NULL age
NOT IN with a NULL	Returns ZERO ROWS. The single most famous SQL trap. Use NOT EXISTS instead. SELECT COUNT(*) AS with_not_in FROM departments WHERE dept_id NOT IN (SELECT dept_id FROM employees) → 0

27 · Interview patterns

Twelve shapes that cover most of what gets asked. Learn the shape, not the query.

Nth highest value	DENSE_RANK = N. Ask first whether ties share a rank, and whether N is by person or by distinct value. WITH r AS (SELECT DISTINCT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rk FROM employees) SELECT salary FROM r WHERE rk=3 → 185000.00
Top N per group	Rank inside a CTE partitioned by the group, then filter rn <= N outside. WITH r AS (SELECT dept_id, emp_name, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn FROM employees WHERE dept_id IS NOT NULL) SELECT dept_id, emp_name FROM r WHERE rn<=2 ORDER BY dept_id → dept_id=1 emp_name=Priya Nair · dept_id=1 emp_name=Ananya Gupta ... 6 rows

Find duplicates	GROUP BY the business key, HAVING COUNT(*) > 1. <pre>SELECT LOWER(raw_email) AS email, COUNT(*) AS copies FROM staging_signups GROUP BY 1 HAVING COUNT(*)>1 ORDER BY copies DESC</pre> → email=ops@zenith.example copies=3 · email=team@orbit.example copies=2 ... 3 rows
De-duplicate, keep latest	ROW_NUMBER partitioned by the key, ordered by recency; keep rn = 1. RANK would leave ties behind. <pre>WITH r AS (SELECT raw_email, loaded_at, ROW_NUMBER() OVER (PARTITION BY LOWER(raw_email) ORDER BY loaded_at DESC) AS rn FROM staging_signups) SELECT COUNT(*) AS unique_rows FROM r WHERE rn=1</pre> → 6
Running total	SUM(x) OVER (ORDER BY d) — cumulative by default. <pre>WITH m AS (SELECT DATE_TRUNC('month',order_date)::date AS mth, COUNT(*) AS n FROM orders GROUP BY 1) SELECT mth, n, SUM(n) OVER (ORDER BY mth) AS cumulative FROM m ORDER BY mth</pre> → mth=2024-01-01 n=6 cumulative=6 · mth=2024-02-01 n=7 cumulative=13 ... 6 rows
Moving average	AVG(x) OVER (ORDER BY d ROWS BETWEEN 2 PRECEDING AND CURRENT ROW). <pre>WITH m AS (SELECT DATE_TRUNC('month',order_date)::date AS mth, COUNT(*) AS n FROM orders GROUP BY 1) SELECT mth, n, ROUND(AVG(n) OVER (ORDER BY mth ROWS BETWEEN 2 PRECEDING AND CURRENT ROW),2) AS ma3 FROM m ORDER BY mth</pre> → mth=2024-01-01 n=6 ma3=6.00 · mth=2024-02-01 n=7 ma3=6.50 ... 6 rows
Month-over-month growth	LAG for the previous period, NULLIF to guard the division. <pre>WITH m AS (SELECT DATE_TRUNC('month',order_date)::date AS mth, COUNT(*) AS n FROM orders GROUP BY 1) SELECT mth, n, ROUND(100.0*(n-LAG(n) OVER (ORDER BY mth))/NULLIF(LAG(n) OVER (ORDER BY mth),0),1) AS pct FROM m ORDER BY mth</pre> → mth=2024-01-01 n=6 pct=NULL · mth=2024-02-01 n=7 pct=16.7 ... 6 rows
Pivot rows into columns	Conditional aggregation. PostgreSQL has no PIVOT keyword — FILTER or CASE is the pivot. <pre>SELECT country, COUNT(*) FILTER (WHERE status='completed') AS completed, COUNT(*) FILTER (WHERE status='cancelled') AS cancelled FROM orders o JOIN customers c USING (cust_id) GROUP BY country ORDER BY country</pre> → country=Germany completed=4 cancelled=0 · country=India completed=19 cancelled=3 ... 5 rows
Gaps and islands	date – ROW_NUMBER() is CONSTANT inside a consecutive run. Group by that constant. <pre>WITH d AS (SELECT DISTINCT event_at::date dt FROM web_events WHERE cust_id=1 AND event_type='page_view'), g AS (SELECT dt, dt-(ROW_NUMBER() OVER (ORDER BY dt)::int) AS island FROM d) SELECT MIN(dt) AS run_start, MAX(dt) AS run_end, COUNT(*) AS days FROM g GROUP BY island ORDER BY run_start</pre> → run_start=2024-03-01 run_end=2024-03-05 days=5 · run_start=2024-03-09 run_end=2024-03-11 days=3 ... 3 rows
Funnel conversion	COUNT(DISTINCT id) FILTER per stage, then divide with NULLIF guards. <pre>SELECT COUNT(DISTINCT session_id) FILTER (WHERE event_type='page_view') AS viewed, COUNT(DISTINCT session_id) FILTER (WHERE event_type='purchase') AS purchased FROM web_events</pre> → viewed=45 purchased=4
Fill date gaps	Build a spine with generate_series and LEFT JOIN the aggregate ONTO the spine, not the other way round. <pre>WITH s AS (SELECT g::date AS mth FROM generate_series(DATE '2024-01-01',DATE '2024-06-01',INTERVAL '1 month') g), a AS (SELECT DATE_TRUNC('month',order_date)::date AS mth, COUNT(*) AS n FROM orders WHERE channel='partner' GROUP BY 1) SELECT s.mth, COALESCE(a.n,0) AS partner_orders FROM s LEFT JOIN a ON a.mth=s.mth ORDER BY s.mth</pre> → mth=2024-01-01 partner_orders=3 · mth=2024-02-01 partner_orders=3 ... 6 rows

Employees earning above their manager

Self-join on manager_id, then compare the two salaries.

```
SELECT e.emp_name AS employee, e.salary, m.emp_name AS manager, m.salary AS mgr_salary FROM employees e JOIN employees m ON m.emp_id=e.manager_id WHERE e.salary>m.salary
→ employee=Ananya Gupta salary=155000.00 manager=Divya Suresh mgr_salary=150000.00
```

28 · Dialect differences

If the interviewer switches engines mid-question, this is the table you want in your head.

TASK	POSTGRESQL	MYSQL	SQL SERVER
Limit rows	LIMIT 10 OFFSET 5	LIMIT 5, 10	OFFSET 5 ROWS FETCH NEXT 10 ROWS ONLY
Top N	LIMIT 10	LIMIT 10	SELECT TOP 10
Concatenate	a b	CONCAT(a,b)	a + b
NULL fallback	COALESCE(a,b)	IFNULL(a,b)	ISNULL(a,b)
Current time	NOW()	NOW()	GETDATE()
Month bucket	DATE_TRUNC('month',d)	DATE_FORMAT(d,'%Y-%m-01')	DATEFROMPARTS(YEAR(d),MONTH(d),1)
Extract year	EXTRACT(YEAR FROM d)	YEAR(d)	YEAR(d)
Date add	d + INTERVAL '1 day'	DATE_ADD(d,INTERVAL 1 DAY)	DATEADD(day,1,d)
Date diff (days)	a - b	DATEDIFF(a,b)	DATEDIFF(day,b,a)
Cast	x::int / CAST	CAST(x AS SIGNED)	CAST / CONVERT
Case-insensitive LIKE	ILIKE	LIKE (collation)	LIKE (collation)
String aggregate	STRING_AGG(x,',')	GROUP_CONCAT(x)	STRING_AGG(x,',')
Upsert	ON CONFLICT DO UPDATE	ON DUPLICATE KEY UPDATE	MERGE
Auto id	GENERATED AS IDENTITY	AUTO_INCREMENT	IDENTITY(1,1)
Quote identifier	"col"	`col`	[col]
FULL OUTER JOIN	yes	no – emulate with UNION	yes
Window functions	yes	yes (8.0+)	yes (2012+)
FILTER (WHERE ...)	yes	no – use CASE	no – use CASE
Boolean type	yes	TINYINT(1)	BIT
TRUNCATE rollback	yes	no	yes
Default isolation	READ COMMITTED	REPEATABLE READ	READ COMMITTED
Regex match	~ / ~*	REGEXP	LIKE / PATINDEX
LATERAL	LATERAL	LATERAL (8.0.14+)	CROSS APPLY
GROUPING SETS	yes	WITH ROLLUP only	yes